EX12 - Vertical Motion with Functions

From a linear script to multiple collaborating functions

1. Learning goal

So far we have written relatively independent functions. In EX12 we will take the Euler vertical-motion simulator from EX08 and split it into small responsibilities. The main goal is not to improve the physics: it is to learn how to design a program using several functions that call each other.

Core idea: each function should have one clear responsibility and a simple interface.

- Separate validation, numerical integration, analytical solution, and errors.
- Avoid hidden global variables.
- Practice multiple return values.
- Understand how one function can call another.
- Prepare for modules and larger projects.

2. Physical model

We use exactly the same model as EX08:

```
dh/dt = v
dv/dt = -g
```

Using Explicit Euler:

```
h_new = h + v * dt
v_new = v - g * dt
t_new = t + dt
```

The analytical solution will still be used to validate the simulator:

```
h_max = h0 + v0**2 / (2*g)
t_f   = (v0 + sqrt(v0**2 + 2*g*h0)) / g
v_f   = -sqrt(v0**2 + 2*g*h0)
```

3. Program architecture

validate_inputs(...)	euler_step(...)	simulate_vertical_motion(...)	analytical_solution(...)	calculate_errors(...)
inputs -> valid/invalid	state k -> state k+1	repeat steps until impact	exact reference values	numerical vs exact

You do not have to use exactly these names, but the problem should be divided by responsibility. One possible sequence is:

```
main
  -> validate_inputs(...)
  -> simulate_vertical_motion(...)
       -> euler_step(...) many times
  -> analytical_solution(...)
  -> calculate_errors(...)
  -> print results
```

Important: simulate_vertical_motion() can call euler_step(). One function can use another function just like it uses math.sqrt().

4. Suggested function responsibilities

Function	Inputs	Outputs	Responsibility
validate_inputs	h0, v0, g, dt	bool / None	Check physical and numerical conditions
euler_step	h, v, t, g, dt	h_new, v_new, t_new	Advance exactly one step
simulate_vertical_motion	h0, v0, t0, g, dt	t_f, max_h, v_f, iterations	Run the complete while loop
analytical_solution	h0, v0, g	h_max, t_f, v_f	Compute exact reference
calculate_errors	numerical, analytical	errors	Compare results

5. One step: the cleanest function

euler_step() is an ideal function for learning design: it does not need a while loop, it does not print, and it does not know the full trajectory. It only transforms the current state into the next state.

```
def euler_step(h, v, t, g, dt):
    # calculate next state
    ...
    return h_new, v_new, t_new
```

This can be read as a mathematical function:

(h_k, v_k, t_k, g, dt) -> (h_{k+1}, v_{k+1}, t_{k+1})

Notice that it returns three values. You already know how to unpack them:

```
h, v, t = euler_step(h, v, t, g, dt)
```

6. Simulation function

simulate_vertical_motion() will be the higher-level function. It initializes the state, runs the while loop, calls euler_step(), updates max_h and iterations, and returns the final results.

```
def simulate_vertical_motion(h0, v0, t0, g, dt):
    # initialize state
    # while altitude >= 0:
    #     call euler_step(...)
    #     update max height
    #     update iterations
    # return results
```

This function should not compute the analytical solution. That is a different responsibility.

7. Main program and data flow

The main program should start looking like a high-level recipe, not a collection of equations:

```
define inputs
validate inputs
run numerical simulation
compute analytical solution
compute errors
print results
```

When main can be read almost like pseudocode, responsibilities are usually separated well.

8. Test cases

Case	h0	v0	dt	Expected
Nominal	0	50	0.1	Valid simulation
Smaller dt	0	50	0.01	Lower Euler error
Initial altitude	100	50	0.1	General analytical formulas
Invalid dt	0	50	0	Rejected
Invalid g	0	50	0.1	Rejected

9. Exercise requirements

• Do not use global variables inside the functions.

• Do not use scipy or RK4.

• euler_step() must perform exactly one step.

• simulate_vertical_motion() must contain the while loop.

• The analytical solution must live in a separate function.

• Keep return None or another clear policy for invalid inputs.

• The final output must compare numerical vs analytical results.

• Debug prints are fine during development, but calculation functions should not depend on them.

10. Learning tracker

Concept	EX12
Multiple collaborating functions	YES
Function calling another function	YES
Separation of responsibilities	YES
Multiple return values	YES
Local scope	YES
High-level main program	YES
Euler reuse	YES
Preparation for modules	YES

Success criterion: if you can read main without seeing the internal equations and still understand the complete program flow, the exercise is well structured.